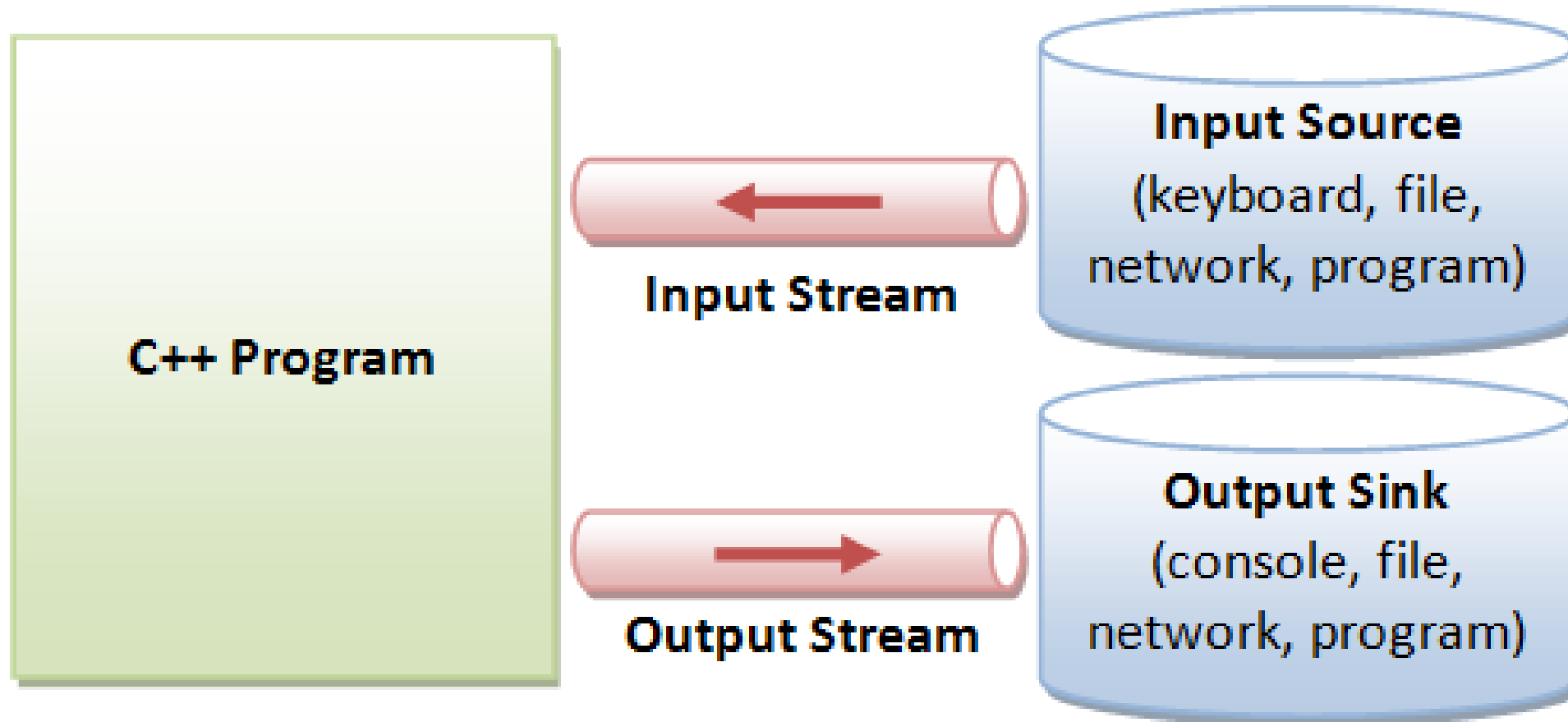


I/O Streams and Exception handling

Streams: Formatted and unformatted data – Manipulators - Files: Opening and Closing a file - File modes - File pointers and their manipulation, Sequential access to a file - Random access to a file - Reading and Writing files, Exception handling: Exception handling constructs - Handling exceptions.

Stream I/O:



Stream I/O:

- C++ **streams** provide a way to perform **input and output (I/O) operations** efficiently. The **iostream** library includes several stream classes for reading and writing data.

Types of Streams in C++:

- C++ provides different types of streams based on their functionality:

Stream	Description
<code>cin</code>	Standard input stream (keyboard input)
<code>cout</code>	Standard output stream (console output)
<code>cerr</code>	Standard error stream (unbuffered error output)
<code>clog</code>	Standard error stream (buffered error output)
<code>ifstream</code>	Input file stream (used to read files)
<code>ofstream</code>	Output file stream (used to write files)
<code>fstream</code>	File stream (for both reading and writing)

Formatted and Unformatted Data in C++ Streams:

C++ provides two types of input/output (I/O) operations using **streams**:

- **Formatted I/O** → Applies specific formatting using manipulators.
- **Unformatted I/O** → Reads/writes data as-is, without additional formatting.

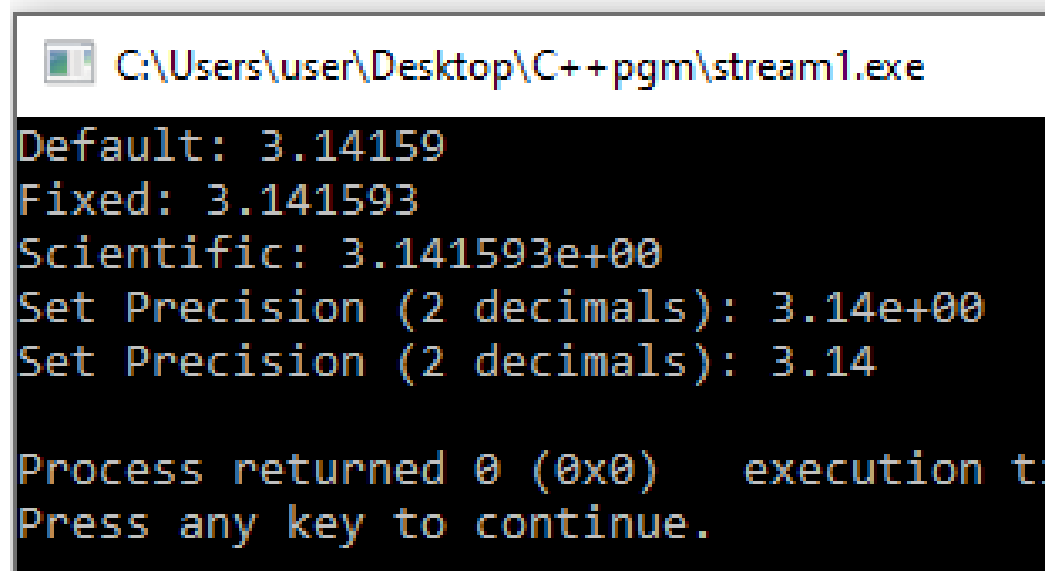
Formatted Input/Output:

What is Formatted I/O?

- Formats the data before displaying or reading.
- Uses manipulators (like `setw`, `setprecision`, `fixed`).
- Helps in aligning and structuring the output.

Example of Formatted Output:

```
#include <iostream>
#include <iomanip> // Required for manipulators
using namespace std;
int main()
{
    double pi = 3.1415926535;
    cout << "Default: " << pi << endl;
    cout << "Fixed: " << fixed
    << pi << endl;
    cout << "Scientific: "
    << scientific << pi << endl;
    cout << "Set Precision (2 decimals): "
    << setprecision(2) << pi << endl;
    cout << "Set Precision (2 decimals): "
    << setprecision(2) << fixed << pi << endl;
    return 0;
}
```



```
C:\Users\user\Desktop\C++pgm\stream1.exe
Default: 3.14159
Fixed: 3.141593
Scientific: 3.141593e+00
Set Precision (2 decimals): 3.14e+00
Set Precision (2 decimals): 3.14

Process returned 0 (0x0)   execution time:
Press any key to continue.
```

Unformatted Input/Output:

What is Unformatted I/O?

- Reads and writes raw data (without any formatting).
- Faster than formatted I/O.
- Uses functions like `get()`, `put()`, `read()`, and `write()`.

Example Using get() and put():

```
#include <iostream>
using namespace std;
int main()
{
    char ch;
    cout << "Enter a character: ";
    ch = cin.get(); // Reads a single character
    cout.put(ch);   // Outputs the character
    return 0;
}
```

C:\Users\user\Desktop\C++pgm\stream2.exe

Enter a character: hai

h

Process returned 0 (0x0) execution

Press any key to continue.

Differences Between Formatted and Unformatted I/O:

Feature	Formatted I/O	Unformatted I/O
Data Formatting	Applies formatting rules	No formatting applied
Flexibility	Customizable (precision, alignment, etc.)	Direct data handling
Speed	Slower (due to processing)	Faster
Functions Used	<code>setw()</code> , <code>setprecision()</code> , <code>fixed</code>	<code>get()</code> , <code>put()</code> , <code>read()</code> , <code>write()</code>

Manipulators:

Manipulators in C++ are functions used to modify the formatting of input and output streams. They help in controlling **alignment, precision, width, and fill characters** in formatted output.

Types of Manipulators in C++:

Manipulators can be categorized into:

- Standard Manipulators (like endl, setw, setprecision)
- User-defined Manipulators (custom manipulators)

Standard Manipulators in C++:

Manipulator	Description	Header File
<code>endl</code>	Inserts a newline (<code>\n</code>) and flushes the buffer	<code><iostream></code>
<code>flush</code>	Flushes the output buffer	<code><iostream></code>
<code>ws</code>	Extracts leading whitespace from input	<code><iostream></code>
<code>setw(n)</code>	Sets the width of output to <code>n</code> characters	<code><iomanip></code>
<code>setfill(ch)</code>	Fills empty spaces with character <code>ch</code>	<code><iomanip></code>
<code>setprecision(n)</code>	Sets decimal precision to <code>n</code> digits	<code><iomanip></code>
<code>fixed</code>	Displays floating-point numbers in fixed-point notation	<code><iomanip></code>
<code>scientific</code>	Displays floating-point numbers in scientific notation	<code><iomanip></code>
<code>hex</code>	Prints integer in hexadecimal format	<code><iostream></code>
<code>oct</code>	Prints integer in octal format	<code><iostream></code>
<code>dec</code>	Prints integer in decimal format (default)	<code><iostream></code>

Standard Manipulators in C++:

<code>showbase</code>	Displays the base (<code>0x</code> for hex, <code>0</code> for octal)	<code><iostream></code>
<code>noshowbase</code>	Hides base prefix	<code><iostream></code>
<code>showpos</code>	Shows a <code>+</code> sign before positive numbers	<code><iostream></code>
<code>noshowpos</code>	Hides the <code>+</code> sign (default)	<code><iostream></code>
<code>boolalpha</code>	Displays boolean values as <code>true/false</code> instead of <code>1/0</code>	<code><iostream></code>
<code>noboolalpha</code>	Displays boolean values as <code>1/0</code>	<code><iostream></code>
<code>uppercase</code>	Displays hexadecimal letters in uppercase (A-F)	<code><iostream></code>
<code>lowercase</code>	Displays hexadecimal letters in lowercase (a-f) (default)	<code><iostream></code>
<code>left</code>	Aligns output to the left	<code><iostream></code>
<code>right</code>	Aligns output to the right	<code><iostream></code>
<code>internal</code>	Puts signs (<code>+</code> or <code>-</code>) before padding spaces	<code><iostream></code>

Example 1: setw, setfill, left, right

```
#include <iostream>
#include <iomanip> // Required for manipulators
using namespace std;
int main()
{
    cout << setw(10) << "Hello" << endl;
    // Right-aligned by default
    cout << left << setw(10) << "World" << endl;
    // Left-aligned
    cout << right << setw(10) << "C++" << endl;
    // Right-aligned
    cout << setw(10) << setfill('*') << "Fill"
    << endl; // Fill empty spaces with '*'
    return 0;
}
```

C:\Users\user\Desktop\C++

```
    Hello
World
    C++
*****Fill

Process returned 0 (0x0)
Press any key to continue
```

Example 1: fixed, scientific, setprecision.

```
#include <iostream>
#include <iomanip> // Required for manipulators
using namespace std;
int main()
{
    double pi = 3.1415926535;
    cout << "Default: " << pi << endl;
    cout << "Fixed: " << fixed
    << pi << endl;
    cout << "Scientific: "
    << scientific << pi << endl;
    cout << "Set Precision (2 decimals): "
    << setprecision(2) << pi << endl;
    cout << "Set Precision (2 decimals): "
    << setprecision(2) << fixed << pi << endl;
    return 0;
}
```

C:\Users\user\Desktop\C++pgm\stream1.exe

Default: 3.14159

Fixed: 3.141593

Scientific: 3.141593e+00

Set Precision (2 decimals): 3.14e+00

Set Precision (2 decimals): 3.14

Process returned 0 (0x0) execution time: 0.000 s

Press any key to continue.

Example 3: boolalpha, hex, showbase

```
#include <iostream>
using namespace std;
int main()
{
    bool status = true;
    int num = 255;
    cout << "Without boolalpha: " << status << endl;
    cout << "With boolalpha: " << boolalpha << status << endl;
    cout << "Decimal: " << num << endl;
    cout << "Hexadecimal: " << showbase << hex << num << endl;
    cout << "Octal: " << oct << num << endl;
    return 0;
}
```

C:\Users\user\Desktop\C++pgm\ma

```
Without boolalpha: 1
With boolalpha: true
Decimal: 255
Hexadecimal: 0xff
Octal: 0377
```

```
Process returned 0 (0x0)   e
Press any key to continue.
```


User-Defined Manipulators-Example.

```
#include <iostream>
using namespace std;
// Custom manipulator to print a line separator
ostream& line(ostream& output)
{
    return output << "\n-----\n";
}
int main()
{
    cout << "Hello, C++!" << line;
    cout << "Custom Manipulator Example" << line;
    return 0;
}
```

 C:\Users\user\Desktop\C++pgm\mani3.

Hello, C++!

Custom Manipulator Example

Process returned 0 (0x0) exe

File Handling

- A file is a container in computer storage devices used for storing data.

Why files are needed?

- When a program is terminated, the entire data is lost. Storing in a file will preserve your data even if the program terminates.
- If you have to enter a large number of data, it will take a lot of time to enter them all. However, if you have a file containing all the data, you can easily access the contents of the file using a few commands in C.
- You can easily move your data from one computer to another without any changes.

File Operations:

- Creating a new file
- Opening an existing file
- Closing a file
- Reading from and writing information to a file

fstream library

In C++, **fstream** library is used to handle files, which provides three main classes:

- **ofstream** (Output File Stream) – Used for writing to files.
- **ifstream** (Input File Stream) – Used for reading from files.
- **fstream** (File Stream) – Used for both reading and writing.

Example:

- `#include<iostream>`
- `#include<fstream>`

File Operations in C++

C++ provides us with four different operations for file handling. They are:

- **open()** – This is used to create a file.
- **read()** – This is used to read the data from the file.
- **write()** – This is used to write new data to file.
- **close()** – This is used to close the file.

Opening files in C++:

- To read or enter data to a file, we need to open it first. This can be performed with the help of 'ifstream' for reading and 'fstream' or 'ofstream' for writing or appending to the file.

Syntax:

- `open( FileName , Mode );`

Here:

- **FileName** – It denotes the name of file which has to be opened.
- **Mode**- different file modes used.

File modes :

- `ios::in` → Open for reading
- `ios::out` → Open for writing (deletes existing content)
- `ios::app` → Open for appending (keeps existing content)
- `ios::trunc` → Truncate file (clear content if exists)
- `ios::ate` → Opens a file and moves the cursor to the end (allows reading from anywhere)
- `ios::binary` → Open in binary mode

File pointers and their manipulation:

- File pointers in C++ are used to keep track of the current position in a file when performing input or output operations using file streams.
- File Pointers:
- There are two types of file pointers in C++:
- get pointer (seekg, tellg) – used for input (reading).
- put pointer (seekp, tellp) – used for output (writing).

tellg() and tellp():

- Returns the current position of the get or put pointer in the file.
- Returns a value of type `std::streampos`.

```
std::ifstream fin("example.txt");
```

```
std::streampos pos = fin.tellg(); // get current read position
```

```
std::ofstream fout("example.txt");
```

```
std::streampos pos = fout.tellp(); // get current write position
```

seekg() and seekp()

- Moves the get or put pointer to a specific location in the file.

seekg(offset, direction);

seekp(offset, direction);

Directions:

- ios::beg – beginning of the file
- ios::cur – current position
- ios::end – end of the file

Example:

- `fin.seekg(0, std::ios::beg);` // move get pointer to beginning
- `fin.seekg(10, std::ios::cur);` // move get pointer 10 bytes ahead
- `fin.seekg(-5, std::ios::end);` // move 5 bytes before end

Example1:

 C:\Users\user\Desktop\C++pgm\file2.exe

```
Read from 7th byte: world!  
Process returned 0 (0x0)   execu  
Press any key to continue.
```

```
#include <iostream>  
#include <fstream>  
int main()  
{  
    std::ofstream fout("data.txt");  
    fout << "Hello, world!";  
    fout.close();  
    std::ifstream fin("data.txt");  
    // Move get pointer to the 7th byte  
    fin.seekg(7, std::ios::beg);  
    char buffer[20];  
    fin.getline(buffer, 20);  
    std::cout << "Read from 7th byte: "  
    << buffer << std::endl;  
    fin.close();  
    return 0;  
}
```

Example2:

```
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
    fstream file;
    // Step 1: Open file for read/write and write something
    file.open("sample.txt", ios::in | ios::out | ios::trunc);
    if (!file)
    {
        cerr << "Failed to open file!" << endl;
        return 1;
    }

    // Writing to file
    file << "Hello, world!";
    // Get current write position
    streampos writePos = file.tellp();
```

```
cout << "Write pointer is at position: " << writePos << endl;
// Step 2: Move the write pointer back to position 7 (after "Hello, ")
file.seekp(7, ios::beg);
file << "C++"; // Overwrites "wor" with "C++"
// Step 3: Move read pointer to beginning
file.seekg(0, ios::beg);
streampos readPos = file.tellg();
cout << "Read pointer is at position: " << readPos << endl;
// Step 4: Read and display modified content
string content;
getline(file, content);
cout << "File content: " << content << endl;
file.close();
return 0;
}
```



C:\Users\user\Desktop\C++pgm\file3.exe

```
Write pointer is at position: 13
Read pointer is at position: 0
File content: Hello, C++ld!
```

Sequential Access:

- In **sequential access**, data is read or written **in order**, from the start to the end.

Example:

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;
int main()
{
    ofstream out("sample.txt");
    out << "Apple\nBanana\nCherry\n";
    out.close();
    ifstream in("sample.txt");
    string line;
    cout << "Reading file sequentially:" << endl;
    while (getline(in, line))
    {
        cout << line << endl; // Reads line by line, in order
    }
    in.close();
    return 0;
}
```

C:\Users\user\Desktop\C++pgm\file4.exe

```
Reading file sequentially:
Apple
Banana
Cherry
```


Random Access:

- In **random access**, you can jump directly to **any position** in the file.

`seekg(position)` → to move the read pointer (get)

`seekp(position)` → to move the write pointer (put)

`tellg()` / `tellp()` → to get current pointer position

Example:

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;
int main()
{
    // Create file with sample text
    fstream file;
    // Step 1: Open file for read/write and write something
    file.open("sample10.txt", ios::in | ios::out);
    if (!file)
    {
        cerr << "File not found!" << endl;
        return 1;
    }

    // Jump to 7th byte (start of "Banana") and read a line
    file.seekg(7, ios::beg); // skip "Apple\n"
    string line;
    getline(file, line);
    cout << "Randomly accessed line: " << line << endl;
```

```
// Go to beginning and overwrite "Apple" with "Mango"
file.seekp(0, ios::beg);
file << "Mango";
file.close();
return 0;
}
```



C:\Users\user\Desktop\C++pgm\file5.exe

Randomly accessed line: Banana

Process returned 0 (0x0) execution time: 0.000 s

Press any key to continue.

Exception Handling:

- **Exception handling** in C++ is a way to deal with **runtime errors** gracefully—without crashing the program.

Exception Handling Constructs:

C++ uses the following keywords:

- **try** → Defines a block of code to test for errors
- **throw** → Throws an exception
- **catch** → Catches and handles the thrown exception

- **Syntax:**

```
try
```

```
{
```

```
    // Code that may throw an exception
```

```
    throw some_exception;
```

```
}
```

```
catch (type_of_exception var)
```

```
{
```

```
    // Code to handle the exception
```

```
}
```

Example:

```
#include <iostream>
using namespace std;
int main()
{
    int a,b;
    cout<<"Enter a and b value one by one.\n";
    cin>>a>>b;
    try
    {
        if (b == 0)
            throw "Division by zero error!";
        cout << "Result: " << a / b << endl;
    }
    catch (const char* msg)
    {
        cerr << "Exception caught: " << msg << endl;
    }

    return 0;
}
```

C:\Users\user\Desktop\C++pgm\exc.exe

Enter a and b value one by one.

5

0

Exception caught: Division by zero error!

Process returned 0 (0x0) execution time

Press any key to continue.

Enter a and b value one by one.

10

5

Result: 2

Process returned 0 (0x0) execution

Press any key to continue.

Example:File Handling with Exception Handling

```
#include <iostream>
#include <fstream>
#include <string>
#include <exception>
using namespace std;
int main()
{
    ifstream file;
    string filename;
    cout<<"Enter file name:(with .txt)\n";
    cin>>filename;
    try
    {
        file.open(filename);
        // Check if file failed to open
        if (!file.is_open())
        {
            throw runtime_error("Error: Unable to open file: " + filename);
        }
    }
```

```
,  
    string line;  
    cout << "Reading file content:\n";  
    while (getline(file, line))  
    {  
        cout << line << endl;  
    }  
    file.close();  
}  
catch (const exception& e)  
{  
    cerr << "Exception caught: " << e.what() << endl;  
}  
return 0;  
}
```


 C:\Users\user\Desktop\C++pgm\filexc.exe

```
Enter file name:(with .txt)
sample10.txt
Reading file content:
Mango
Banana
Cherry
```

 C:\Users\user\Desktop\C++pgm\filexc.exe

```
Enter file name:(with .txt)
sample10
Exception caught: Error: Unable to open file: sample10

Process returned 0 (0x0)   execution time : 4.688 s
Press any key to continue.
```